

Task1

- 1.1 For this practical we have decided to create a music festival called Sonic Horizons Festival. This is a large outdoor festival that hosts live performances, food vendors and multiple types of operational services. This festival is divided into several operational areas that work together to provide entertainment, safety and top-notch services. Event organisers should be able to coordinate these areas effectively, especially when responding to weather conditions, capacity limits as well as emergency situations. We have made use of hierarchies so that the entire area can be managed uniformly. This will allow the festival management to perform operations on a complete section without needing to interact directly with every unit.

Event Structure

SonicHorizonsFestival

```
|
|----- PerformanceZone
|   |---- MainStageArea
|       |---- MainStage
|       |---- BackStage
|   |---- ElectronicStageArea
|       |---- DjStage
|       |---- DanceFloor
|--- FoodCourt
|   |--- BurgerVendor
|   |--- BeverageVendor
|--- OperationsZone
|   |--- SecurityPost
|   |--- FirstAidTent
|
```

- 1.2 **Observer:**

Subject: FestivalSubject

ConcreteSubject: FestivalControl, FestivalGroup

Observer: FestivalObserver,

ConcreteObserver: FestivalUnit, FestivalGroup

FestivalControl generates notices and distributes them through the Observer pattern. FestivalGroup objects receive notices from parent subjects and forward them to lower-level observers. FestivalUnit objects receive notices and react according to their individual responsibilities.

Composite:

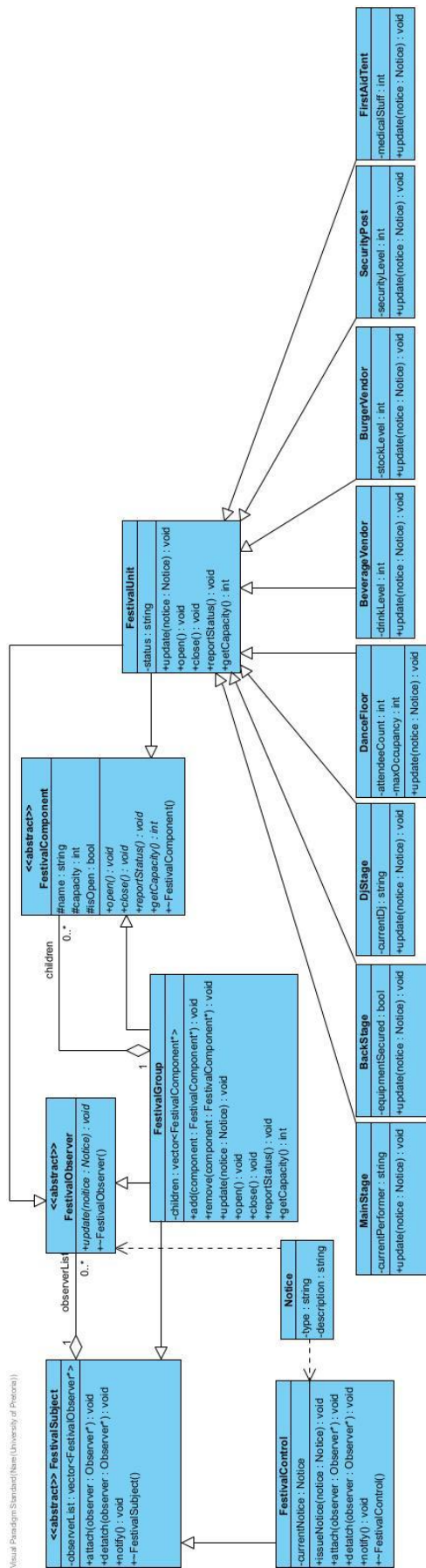
Component: FestivalComponent

Leaf: FestivalUnit

Composite: FestivalGroup

ConcreteLeaves: MainStage, BackStage, DJStage, DanceFloor, BurgerVendor, BeverageVendor, SecurityPost, FirstAidTent

The Composite pattern represents the physical structure of the festival. FestivalGroup objects own FestivalComponent objects, allowing individual units and entire areas to be treated uniformly. FestivalGroup creates the instances such as food court , performance zone and etc as they all share the same behaviours.



1.2.1 a) The FestivalGroup contains FestivalComponents, which can be individual parts (Leaves) or other groups (Composites). Actions like open(), close(), and getCapacity() are passed down to its children. This lets us treat the whole festival and its individual parts in the same way. For example, SonicHorizonsFestival->PerformanceZone-> MainStageArea-> MainStage.

b) Direct calls would make FestivalControl depend on every festival unit. The Observer pattern lets festival units register for alerts and receive notifications automatically. It allows for a one-to-many dependency and reduces tight coupling. For example, a WEATHER_ALERT can notify the MainStage, DJStage, SecurityPost, and FirstAidTent without changing FestivalControl.

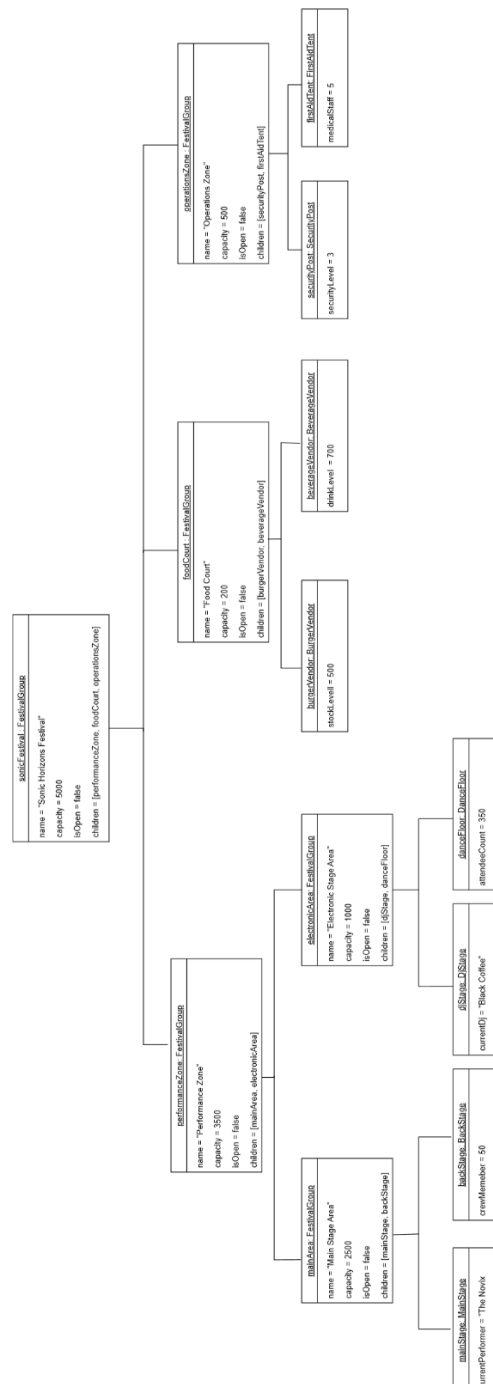
c) Event components are owned exclusively by the FestivalGroup (Composite) that contains them. The root FestivalGroup (SonicHorizonsFestival) is created by the client (our main.cpp) and owns its direct children. Those children, if they are Groups, own their own children, and so on. This creates a clear, acyclic ownership tree where every component has a single owner.

d) Observer references are not owned. The pointers stored in a FestivalSubject's observer's vector are raw pointers (non-owning). The subject only uses them to call update(). The observer's lifetime is managed by its owner in the Composite tree. This separation is vital: it prevents the Subject from accidentally deleting an object that is also owned by its parent, which would lead to a double-deletion bug.

e) A FestivalGroup can be both a Subject and an Observer because it uses each role for a different purpose. As an Observer, it receives notices from a higher-level Subject. As a Subject, it forwards those notices to its own observers. For example, MainStageArea receives a notice from PerformanceZone and then forwards it to MainStage and Backstage. Therefore, being both a Subject and Observer is not a misuse because each role has a different purpose in the collaboration.

Task 2

2.3



2.4 Each FestivalGroup owns its FestivalComponent children. The root SonicHorizonsFestival therefore owns the complete festival hierarchy through its child groups. When the root is destroyed, its destructor deletes its children. If a child is another FestivalGroup, its destructor also deletes its children. This continues

recursively until all leaves are destroyed. Since each component has only one owner, every object is deleted exactly once. If a component is transferred between groups, `remove()` removes it from the original group without deleting it and returns the pointer. The new group then takes ownership when `add()` is called. This prevents memory leaks and double deletion during transfers.

Task 5

Sequence Diagrams 1 through 5

